

IT UNIVERSITY OF CPH

First Year Project

Projects in Data Science

Friday May 29, 2026

Group: Lizards

Hunor Szabó
husz@itu.dk

Simon Friland
simfr@itu.dk

Viktória Kapicáková
vkap@itu.dk

Marcus Bülow Pedersen
mbyp@itu.dk

Sofie Zimmer Ladevig Pedersen
sozp@itu.dk

This project report is presented for the course:
Projects In Data Science,
2nd semester of BSc, Data Science,
IT University of Copenhagen,
Copenhagen, Denmark

Repository:
<https://github.com/SofieITU/2026-PDS-Lizards.git>

Contents

1	Introduction	2
2	Data	2
2.1	Initial Data	2
2.2	Cleaning the Dataset	2
3	Data Exploration	2
3.1	EDA	2
3.1.1	Bar plot	3
3.1.2	Box-plot	3
3.1.3	Correlation Matrix	3
3.1.4	Scatter-plot	4
3.1.5	Random Forest	4
4	Baseline Classification Model	4
4.1	Feature extractions	4
4.1.1	Asymmetry	4
4.1.2	Border compactness	5
4.1.3	Colour	5
5	Evaluation and Results	5
5.1	KNN Classifier	5
5.2	Random Forest Classifier	6
5.3	Solving imbalance with SMOTE (Synthetic Minority Over-sampling Technique)	7
6	Open Question	8
6.1	Histogram of Oriented Gradients	9
6.2	Local Binary Pattern	11
7	Conclusion	13
7.1	What could we improve?	13
8	Appendix	15
8.0.1	Bar plot	15
8.0.2	Box-plot	15
8.0.3	Correlation Matrix	15
8.0.4	Scatter-plot	16
8.0.5	Random Forest	16
8.1	Estimator and max depth on Random Forest Classifier	18

1 Introduction

Skin cancer is among the most commonly diagnosed forms of cancer worldwide. The three primary types are Basal Cell Carcinoma (BCC), Squamous Cell Carcinoma (SCC), and Malignant Melanoma (MEL) [5].

To determine whether a skin lesion is cancerous, patients typically consult a dermatologist. During examination, dermatologists often rely on the ABCDE-criteria: Asymmetry, Border irregularity, Color variation, Diameter, and Evolution, to assess the likelihood of the lesion being cancerous [4].

In recent years, the amount of cases of skin cancer has steadily increased, and it is expected to keep increasing. Despite this, many still remain uncertain about when to seek medical evaluation. Early detection is critical, as it can significantly improve prognosis and, in some cases, determine survival. Increased awareness of skin cancer symptoms, along with advances in diagnostic tools, has contributed to the rising number of detected melanoma cases [3].

In this report, we propose a model designed to extract ABC-based features from skin lesion images. To improve the quality of feature extraction, preprocessing steps such as hair and pen removal are applied, resulting in clearer images and more reliable analysis. Additionally, we investigate the impact of feature extraction techniques such as Histogram of Oriented Gradients (HOG) and Local Binary Patterns (LBP), as well as the use of Synthetic Minority Over-sampling Technique (SMOTE), to evaluate whether these methods enhance classification performance and how they influence the final results.

2 Data

2.1 Initial Data

To train our model, we used the PAD-UFES-20 dataset [7]. The data set contains 2298 images collected from 1373 patients. Each of the images have been labeled with one of the following six skin lesion types: Actinic Keratosis (ACK), Basal Cell Carcinoma of skin (BCC), Malignant Melanoma (MEL), Melanocytic Nevus of Skin (NEV), Squamous Cell Carcinoma (SCC), Seborrheic Keratosis (SEK). The six skin lesion types have been grouped into two general groups:

- Benign: ACK, NEV, SEK
- Cancerous: BCC, MEL, SCC,

The dataset contains approximately 1055 benign and 1048 cancerous cases. The data were collected from smartphone devices, which means that they can be taken in different lighting, angles, distances from the lesion, and image quality.

2.2 Cleaning the Dataset

Looking through all the images and masks, we found that not all images had a mask, or their masks were unfitted. Therefore, we decided to discard all images containing missing or unfitted masks.

By doing this, we ensure that our models will only be trained on consistent and reliable input-label pairs. If an image has no mask, it provides no useful information for learning, and if the mask is incorrect, it introduces noise and potentially misleading patterns. The quality of the dataset improves, when choosing to only keep the properly matched image-mask pair. This typically leads to better model performance, because the model learns from accurate examples instead of being confused by incorrect ones.

Though, removing that data also reduces the size of the dataset. Too small of a dataset can negatively impact the model, since less examples can lead to less information being available for the model to learn from. Bias may also occur, if the removed data are concentrated in certain categories or conditions, making the remaining dataset less representative of real-world variation.

Though generally deleting the images without masks or with incorrect masks will improve the data quality and enhance the models reliability and accuracy. The main compromise is the potential loss of data quantity and diversity.

In total there were 196 missing masks and 6 unfitted masks, which left us with a total of 2096 correctly fitting masks. That means we went from 2298 images down to 2096, that means a loss of 202 images.

3 Data Exploration

3.1 EDA

First of all, we finished our feature extraction and made a pipeline that extracts data from our dataset. We ran 500 pictures through the program and got new data to use. We chose to do some exploratory data analysis on this. For us to be able to see a clear pattern, we had to have enough data, which is why we went with 500 pictures.

Originally we wanted to test it on more than 500 pictures, but our code for feature extractions would take quite a while to run on more than 500 pictures. So we chose 500 as our threshold, since it still produced reasonable enough results.

3.1.1 Bar plot

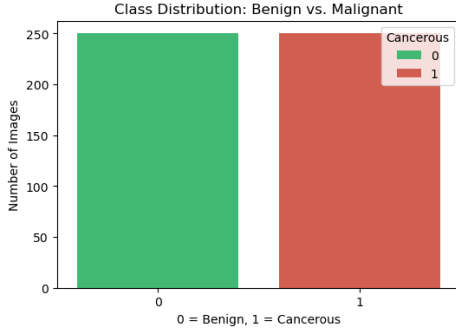


Figure 1: Split between Benign and Cancerous Lesions

The first thing we wanted to look at was the split between how many benign and cancerous lesions there were in our dataset of 500 pictures. It turned out there was a 50/50 split, roughly. This made us wonder if that could also be the case for the rest of the data set. Taking a look into the whole data set, we saw that it turned out to be the case.

3.1.2 Box-plot

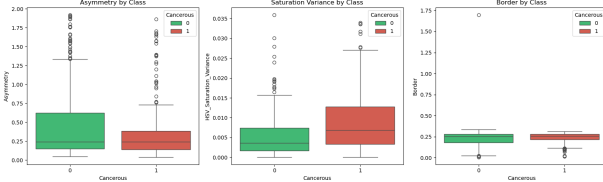


Figure 2: Outliers in Asymmetry, Saturation Variance and Border

Next we wanted to look into the dataset and see if there were any outliers. We only chose to look at Asymmetry, Saturation Variance and Border, because we wanted to get a general idea of how the data looked, while not having to look at all of them.

Looking at the box-plots for Asymmetry and Saturation Variance we saw quite a lot of outliers. Since there were so many outliers in the box-plots and a good chance that there were a lot more in the whole dataset, we chose not to remove them.

Looking at the box-plot for Border there was only one outlier, one which deviated a lot. We chose to remove this one, and going forward exploring the dataset we had to look out for further outliers when extracting all of the data. We wanted to remove outliers like mentioned before, because they can have a big impact on a model, and when there is only one outlier it is definitely not the norm.

3.1.3 Correlation Matrix

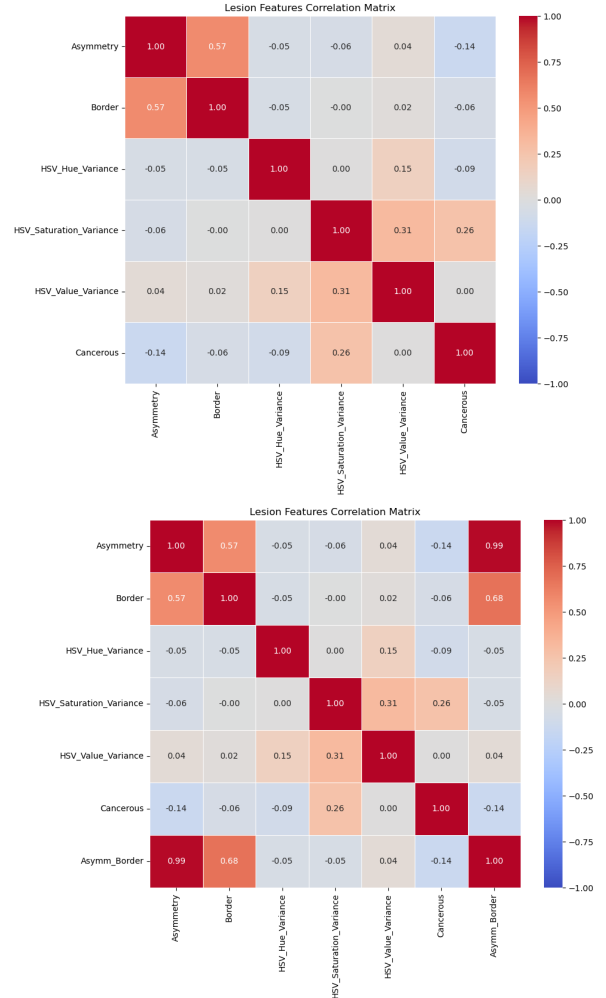


Figure 3: Correlation Matrices with and without Multiplied Asymmetry and Border

We wanted to see if there was any correlation between all the features we chose and cancer. As we can see for most of them there is no significant correlation between the features and cancer except for Saturation Variance, which has the highest correlation with a score of 0.26. To begin with we were honestly a little distressed because of this. We saw that there was good correlation between Border and Asymmetry, and we thought that if we were to combine the two, there was a chance it would end up with correlation with cancer. There was not, but that was the reason why we made two almost identical correlation matrices. Now this is not eccentrically bad when training a machine learning algorithm, depending on which model you train, and how. But it is definitely something we have to take into consideration when training the model, and see if the algorithm that look at linear relationship will be worse then the machine

learning algorithms that do not care about it as much.

3.1.4 Scatter-plot

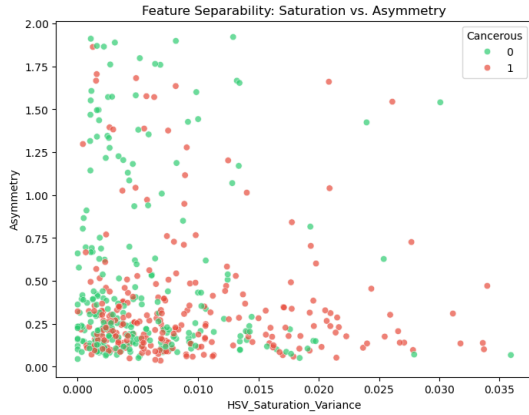


Figure 4: Saturation vs. Asymmetry

After looking at the correlation matrix we quickly wanted to see if there were any noticeable patterns in the data. We chose to look at Saturation vs. Asymmetry in a scatter-plot. As we expected there was not any notable patterns in between the two features we chose. Since there was no correlation in the correlation matrix, we didn't expect to see any pattern. We really only made the scatter-plot for our own sanity's sake so we could get a good visualization and if we look at any of the other scatter-plots we would properly also get what we expect.

3.1.5 Random Forest

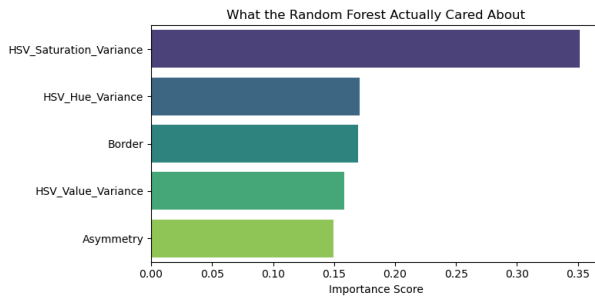


Figure 5: Importance of Features for Random Forest

Since our data might not be the best with a linear machine learning algorithm, we thought that maybe the Random Forest model might be better. We wanted to know which of the features a Random Forest might find most important. In Scikit-Learn there is a way to find out which of our feature that is. As we can

see from the bar-plot above, there is one of the features that stands out. Funnily enough, it is Saturation Variance that has the highest importance score. Saturation Variance also has the highest correlation if we look at the correlation matrix. Random Forest does not care about correlation, and is not the reason Random Forest cares about it. But this suggests that Saturation Variance is one of the strongest and most relevant pieces of data we have.

4 Baseline Classification Model

When building our baseline classification model, we digitalized the clinical practice of a dermatologist. When examining skin lesions, dermatologists typically employ the ABC baseline model, evaluating features such as asymmetry, border irregularity, and color invariance.

With help and inspiration from the code provided in our exercise classes, we developed an environment to automatically screen lesions for these specific ABC traits. Our baseline classification model extracts these named features to determine whether an imaged lesion is cancerous or benign. To achieve the best possible diagnostic accuracy, the features are extracted from pre-processed images where the lesions have been segmented to minimize background noise.

4.1 Feature extractions

4.1.1 Asymmetry

The asymmetry of a skin lesion is one of the first things dermatologists look at when evaluating the possibility of cancer. Clinically, it determines whether a patients skin lesion is asymmetric or not. Technically, however, we have implemented a method to measure the asymmetry, by processing the provided binary masks for all the skin lesions.

First, we find the center of the binary masks. Then we crop out the lesion and sum all non-overlapping pixels by flipping the axes of the mask over the other using the XOR operation. This is done with six rotations of 30 degrees to cover the whole lesion, where the asymmetry score is evaluated by aggregating the sum of intersectional pixels from all rotations divided by the union of pixels across all rotations. Finally, the average is found to produce a rotation-invariant score between 0 (symmetry) and 1 (asymmetry).

We are aware, based on our previous feature box-plots, that minor bugs in our code resulted in an inaccurate distribution of the asymmetry scores. There should not be that many outliers outside the range [0,1], and the score distribution of the benign skin lesions have a significantly higher upper quartile com-

pared to cancerous ones, making us question the feature implementation of asymmetry, since higher asymmetry should indicate cancer.

4.1.2 Border compactness

Secondly dermatologists look at the border of the lesion. Typically cancerous lesions will have a irregular border, and a non cancerous lesion will look closer to a perfect circle. We have used the Polsby-Popper measure on the masks, to determine whether the shape of our lesion is compact or non-compact. To get border variables, we used compactness values:

$$\text{Compactness} = \frac{4\pi \cdot \text{Area}}{\text{Perimeter}^2}$$

The values go from 0 to 1. 0 meaning a highly elongated or irregular shape, and 1 a perfect circle. But statistically it is very unlikely for the value to be exactly 1.

4.1.3 Colour

Colour variation is an important indicator when evaluating skin lesions since cancerous lesions often contain multiple irregular shades and brightness levels. We extracted colour features using the HSV colour space, where hue represents the colour type, saturation represents colour intensity and value represents brightness.

First, the lesion area was segmented using the SLIC superpixel algorithm, which divides the lesion into smaller colour regions while ignoring the background using the binary mask. The image was then converted from RGB to HSV colour space and the mean HSV values were computed for each segment.

Finally, the variance of the hue, saturation and value channels across all segments was calculated. A value close to 0 indicates that the lesion has relatively uniform colouring, intensity or brightness across all segments, while larger values correspond to stronger irregularities and colour variation within the lesion. In particular, hue variance measures changes in colour type, saturation variance measures differences between pale and vivid areas and value variance measures differences between darker and brighter regions.

5 Evaluation and Results

After extracting the lesion features, we proceed to evaluate different machine learning approaches for the the skin lesion classification. Since the extracted features form a relatively low-dimensional feature space, we selected two baseline machine learning models that are suitable for structured tabular data: the K-Nearest

Neighbor(KNN) classifier and the Random Forest classifier.

The purpose of this evaluation was to determine how well the extracted features can distinguish between benign and cancerous skin lesions. In addition, we also paid attention to the number of false negatives (cancerous lesions incorrectly classified as benign). In medical imaging applications such errors are specially critical because they may lead to undetected melanoma cases.

For both models we performed a hyperparameter tuning and validation in order to identify suitable model configuration and to analyze how different parameters affect the classification performance.

5.1 KNN Classifier

[10]The main idea behind this machine learning algorithm is that samples with similar feature values are likely to belong to the same class. In our project, the extracted image features were assymetry, border irregularity and HSV(hue, saturation, value) variance, which make a multi-dimensional feature space. The algorithm classifies a lesion as cancerous/benign by looking at the K closest neighbouring samples in the feature space and deciding the class based on majority voting.

One important parameter of the KNN classifier is the number of neighbours, denoted as K. Small values of K make the model highly sensitive to local patterns and noise, while large values of K smooth the decision boundary too much and may lead to incorrect classifications.

Since our project focuses on medical imaging, minimizing false negatives was one of the most important objectives. False negatives correspond to lesions that are actually cancerous but predicted as benign by the algorithm. We tried to minimize this parameter with maintaining a suitable number for K.

Before training the model, the data set was split into training and testing subsets using an 80/20 train-test split. Stratified splitting was used to preserve the ratio of cancerous and benign samples in both subsets. This creates a more realistic evaluation of the model performance and prevents biased testing results.

To determine the optimal value of K, we performed cross-validation for all odd values for K between 1 and 19. Figure 6 shows the cross-validation results for both accuracy and recall. Accuracy measures the overall proportion of correctly classified samples, while recall measures the proportion of correctly detected cancerous lesions. Recall answer the question: "Out of all cancerous lesions, how many did the model correctly detect?". The formula is:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Where TP = True positives (cancer correctly detected) and FN = False negatives (cancer exists but model missed it). Since recall directly relates to minimizing false negatives, it was considered to be more important than overall accuracy.

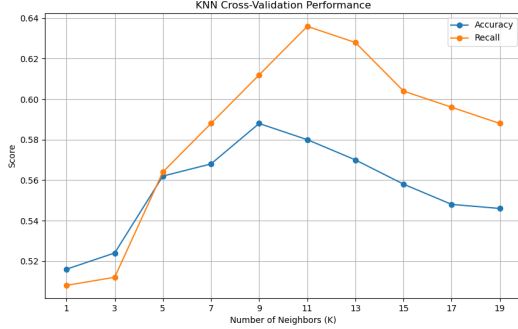


Figure 6: Cross Validation (Accuracy and Recall)

From the validation curve, we observed that the recall score improved as K increased up to approximately 11 neighbors, while the accuracy peaked around K=9. However we also evaluated the number of false negatives directly, as shown in Figure 7, although some larger K values achieved comparable accuracy scored, they produced a higher number of false negatives, which is not good for medical diagnosis tasks.

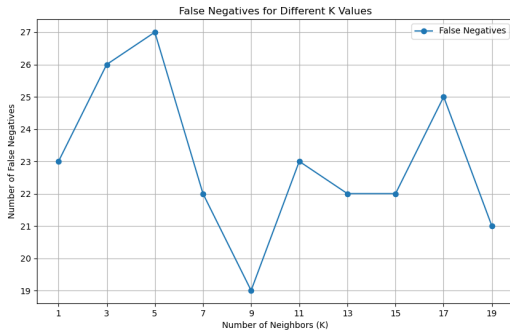


Figure 7: False Negatives for Different K Values

After comparing the false negative counts for different values of K, we selected K = 5 as the final model parameter. Even though some larger values produced slightly better validation accuracy, the model with K = 5 provided more balanced performance and lower false negative count.

The final confusion matrix for the selected model below correctly classified 31 cancerous lesions and 28

benign lesions. At the same time, there were 19 false negatives and 22 false positives. While the number of false positives is relatively high, this behaviour is generally more acceptable in medical imaging than missing cancerous lesions.

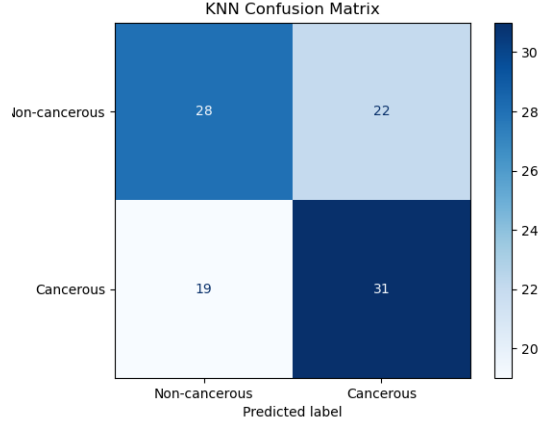


Figure 8: Confusion Matrix of KNN Evaluation

5.2 Random Forest Classifier

[12] Random Forest Classifier models are very popular models for large number of tasks. In short, it's a method to produce aggregated predictions using the predictions from several decision trees. One way to produce multiple decision trees is to train each model with different training sets. This is achieved with Bootstrap Aggregating (or short with Bagging). This way we get multiple different decision trees, with different predictions. For each split of the tree building, we compute the best split using only a randomly selected subset of the features. This is another way to ensure that the decision trees are as different as possible. Each tree comes up with a prediction and the final prediction for the Random Forest Classifier is decided by majority voting. In a general Decision Tree, high variance is very damaging, however in a Random Forest Classifier it is essential, as it helps keeping correlation low between the different trees and thus helping against overfitting.

In our project we decided to run our models on a subset of 500 pictures. After feature extraction and splitting the data into train and test sets, we were very interested in what hyperparameter we need to adjust. There are 2 parameters of interest, number of estimators and max depth. Number of estimator is the number of trees in the forest, and max depth, as the name suggests, is the maximum depth of the tree. If it is set to None, the nodes until all leaves are pure or until all leaves contain less than minimum samples

split size. The question arose, what do we think is the most optimal parameters we can have, thus we wrote a little script using parts of code from exercise classes. We got the following graph:

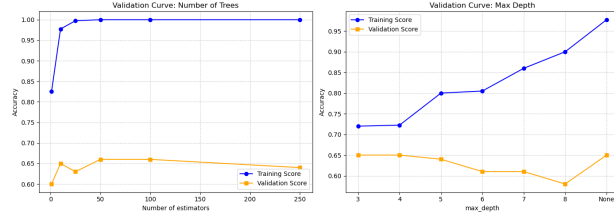


Figure 9: Analysis on number of estimators and max depth

Looking at the plot, we concluded that the most optimal number of estimator is 10, as it is loosing accuracy with more than 10 estimators. Max depth is peeking at 3-4, thus we concluded that the most optimal parameters for our Random Forest Classifier model is 10 number of estimators with max depth either 3 or 4. This yields the highest accuracy all together, however accuracy is not specifically a decisive factor in medical imaging. It is important that the algorithm correctly identifies the pictures, however it happens that the algorithm classifies something benign, when it is actually cancerous (False Negative). We want to minimize that number, therefore we decided to look at the confusion matrices. We ran our model through a loop with different parameters (10, 25 and 50 for number of estimator and 3, 4, 5, 6, 7 for max depth).

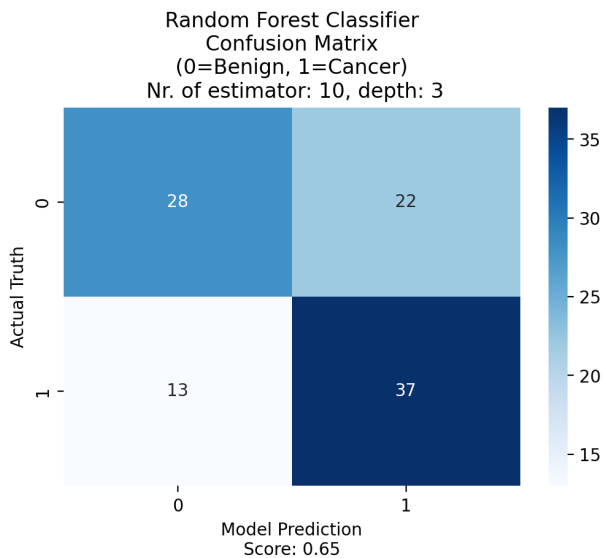


Figure 10: Confusion Matrix fro Random Forest Classifier (Nr. of estimator: 10, depth: 3)

As it turned out it does not look that bad on the false negative side, however we looked at the different confusion matrices where there was one with the same accuracy score, yet it has fewer false negatives. Ultimately, we believe that is a better model because it has less patients who the algorithm predicted healthy, even though they have cancer in reality.

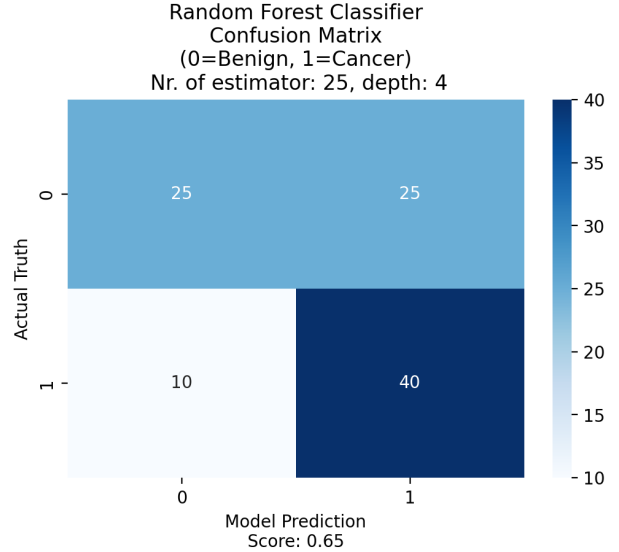


Figure 11: Confusion Matrix for Random Forest Classifier (Nr. of estimator: 25, depth: 4)

Therefore our final parameters for Random Forest Classifier is 25 estimators and 4 max depth.

5.3 Solving imbalance with SMOTE (Synthetic Minority Over-sampling Technique)

[2] Although our dataset is generally balanced, its limited size motivates the use of SMOTE to generate synthetic samples for the minority class; thus, the key question is whether augmenting cancerous mole instances in this way improves the Random Forest's detection performance more than simply applying class-weight penalties.

Currently, we handle the class imbalance by setting `class_weight='balanced'`, which effectively penalizes misclassifications of cancerous images more heavily—for example, missing a cancer case incurs a much higher cost than misclassifying a benign one. SMOTE, however, takes a different approach. Instead of adjusting penalties, it generates additional synthetic examples of the minority class. It does this by identifying existing cancerous data points and creating new, realistic samples along the feature space between them, thereby increasing the representation of cancer

cases until the dataset becomes balanced.

The way SMOTE can improve the Random Forest is by giving it more "dots" to look at, so it can draw smoother and more accurate decision boundaries around the cancer class, which often drastically improving the models performance.

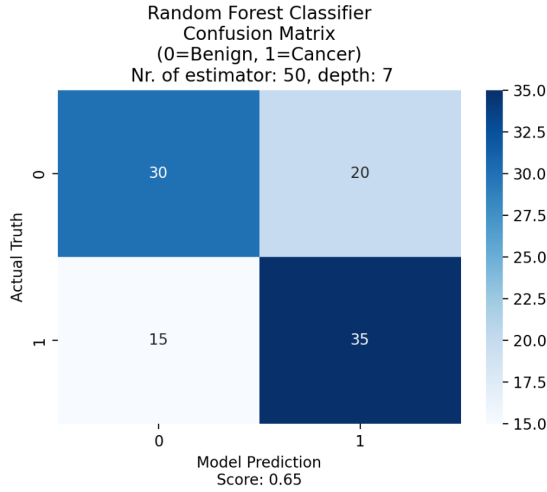


Figure 12: Confusion Matrix for Random Forest Classifier

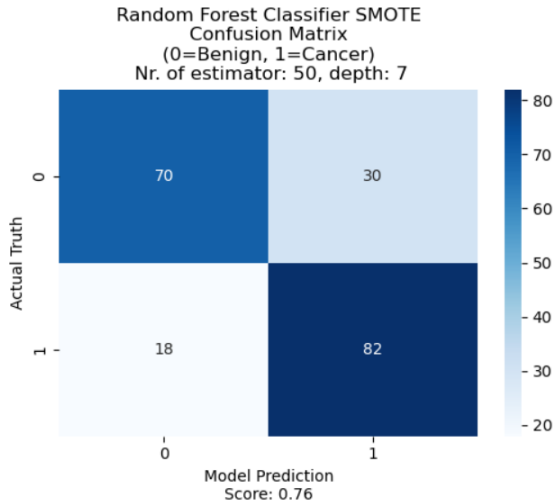


Figure 13: Confusion Matrix for Random Forest Classifier with SMOTE

We made a new features csv file for SMOTE and then combined it with our already existing features csv file. That meant that we now had 1000 data points to work with. As seen above in the two confusion matrices, with SMOTE the model prediction score goes up to 0.76, from the 0.65 from the original. So it does improve the Random Forrest a bit.

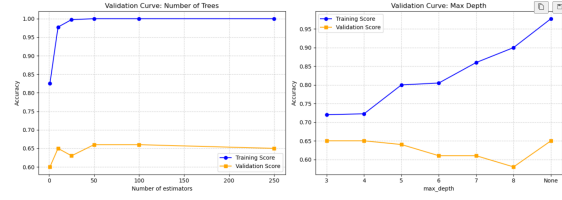


Figure 14: Validation Classifier

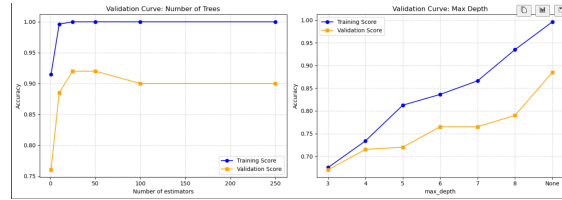


Figure 15: Validation SMOTE

As seen in the two graphs above, the validation score improves significantly from approximately 0.65 to 0.90 after applying SMOTE, indicating that the original dataset likely suffered from class imbalance. In both cases, the training score remains at 1.00, showing that the model is highly flexible but also prone to overfitting. Without SMOTE, the baseline classifier demonstrates strong overfitting and poor generalization performance. However, after handling class imbalance with SMOTE, validation performance improves substantially, and the model is able to benefit from increased complexity. Overall, this suggests that addressing class imbalance is more important than only tuning hyperparameters.

We are suspicious of the 90% accuracy score, and looking more into it, we found that we introduced data leak using SMOTE incorrectly, when introducing SMOTE before splitting the dataset.

6 Open Question

For our open question we decided to investigate the idea of using deep learning model, as it was suggested to us that it could significantly improve performance when set up correctly. However, it would have been too ambitious for us to train a whole Convolutional Neural Network (CNN), so we decided to investigate the stepping stone of Computer Vision and CNN, namely Histogram of Oriented Gradients and Local Binary Pattern. These algorithms allow us to mathematically extract complex, high-dimensional textures, as this approach grants our model the advanced visual capabilities typically reserved for deep learning, while strictly maintaining the transparent feature explainability required for medical diagnostics.

6.1 Histogram of Oriented Gradients

Histogram of Oriented Gradients, or shortly HOG [11], is a feature used in computer vision and image processing for object detection. It captures the shape of the object, thus it is very popular in human and vehicle detection. This method counts occurrences of gradient orientation in localized portion of an image. The essential thought behind the histogram of oriented gradients descriptor is that local object appearance and shape within an image can be described by the distribution of intensity gradients or edge directions. The image is divided into small connected regions called cells, and for the pixels within each cell, a histogram of gradient directions is compiled. There are four main steps of HOG:

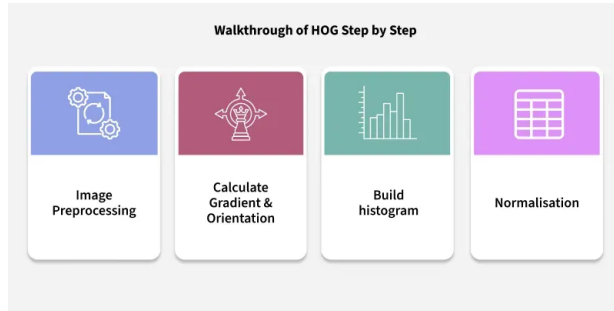


Figure 16: 4 main steps of HOG [1]

- **Step 1:** Pre-processing - converting image to grayscale and resize it to standard size e.g. 32 x 64 pixels. This ensures consistency across all input images, making further analysis and feature extraction more reliable.
- **Step 2:** Calculate Gradient and orientation - Calculate the gradient of the image in both the X and Y directions. For each pixel, calculate gradient magnitude ($\text{Magnitude} = \sqrt{G_x^2 + G_y^2}$) and gradient orientation ($\text{Orientation} = \arctan(G_y, G_x)$).
- **Step 3:** Create orientation histogram - After the previous calculations, the orientation values range between 0 - 180. Divide the 180 degrees range into 9 bins, each spanning 20 degrees. Assign each pixel's gradient magnitude to the two nearest bins, weighted by how close the orientation is to each bin center. Repeat this process for all pixels in the cell, accumulating weighted magnitudes in each bin. The result for each cell is a 1 x 9 feature vector representing the distribution of edge directions.
- **Step 4:** Normalize Across Blocks - Group neighboring cells into blocks, creating overlapping re-

gions as the block moves one cell at a time across the image. For each cell, HOG creates a 1 x 9 feature vector and for each block, concatenate the feature vectors of its four cells to form a 1 x 36 block feature vector. Compute the normalization factor k as the root sum of squares of all 36 block features, then divide each feature in the block by k to obtain the normalized block vector. Repeat this process for all blocks and for last concatenate the normalized vectors from all blocks horizontally to form the final image descriptor.

This high-dimensional descriptor represents the image's edge and texture structure for use in machine learning tasks.

So how does this help our project? We assume that using this feature extraction would improve our Random Forest Classifier. Let us start with visualizing 2 different pictures, one that is benign and one that is cancerous. This could be helpful to visually identify if there are any difference.

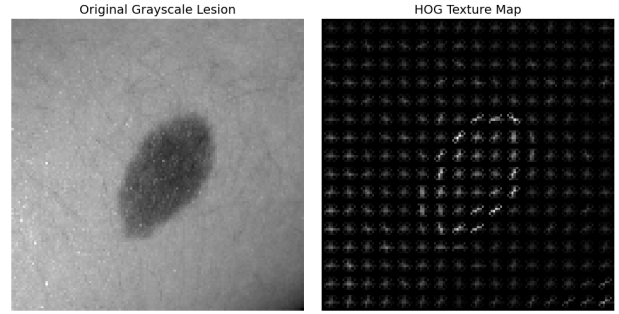


Figure 17: Benign lesion

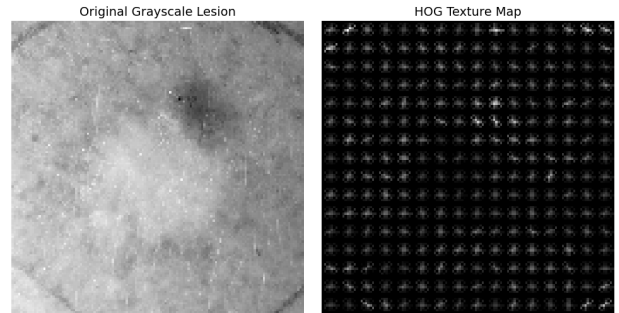


Figure 18: Cancerous lesion

Visually, benign lesions have a nice border outline, meanwhile there is no clear borderline in the cancerous case. Why does this happen? A healthy mole is essentially a cluster of pigment cells that grew in a very orderly, contained way. This results in a sharp,

physical boundary between the mole and the normal skin next to it. Cancerous lesions are characterized by chaotic, uncontrolled cell growth, creating jagged, fuzzy borders.

When HOG looks at a benign mole, it scans the skin, hits that sharp boundary, and sees a sudden jump from light skin pixels to dark mole pixels. Because the edge is relatively smooth, every single HOG arrow aligns perfectly, pointing inward toward the dark center. In contrast, when HOG looks at a cancerous lesion, it cannot find a clean jump. The cancer cells are bleeding into the skin, so the pixels fade from dark brown, to light brown, to tan, to pink.

So how does all this perform in our project? It works very well. First of all, we chose Random Forest Classifier as our model, due to the extreme amount of data points HOG generates. By extracting HOG textures, we introduced over 1,700 new features to our dataset, which could be an issue for other classification methods. Random Forest is uniquely robust against this problem. Because it randomly selects subsets of features for every individual tree, it naturally prevents the model from overfitting on noisy pixels or camera artifacts, which could possibly destroy a simpler model. The two parameters we could change, using Random Forest Classifier is number of estimators and tree max depth. After trying out a few options we get the following plot:

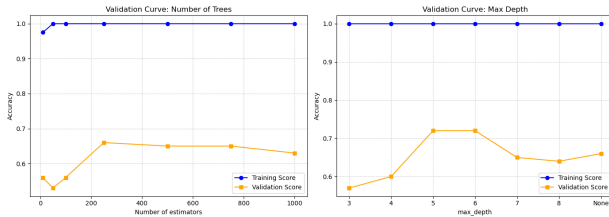


Figure 19: Analysis on number of estimators and max depth

After looking at the 2 plots, we concluded that the sweet spot for number of estimators are 250, as it has the highest score, and either 5 or 6 max depth. In order to find our final hyper parameters, we decided to look at the confusion matrices as well. We were primarily interested in the ones with 250 estimators at max depth 5 and 6:

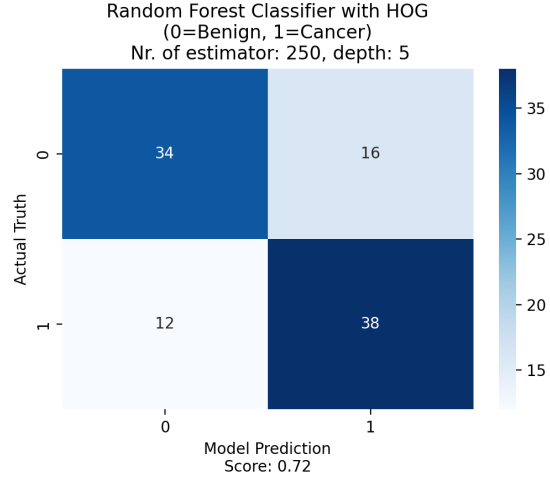


Figure 20: Confusion Matrix for Random Forest Classifier with HOG (Nr. of estimator: 250, depth: 5)

Both models have 0.72 accuracy score, however the one with max depth 6 has a slightly better performance on the False Negative side.

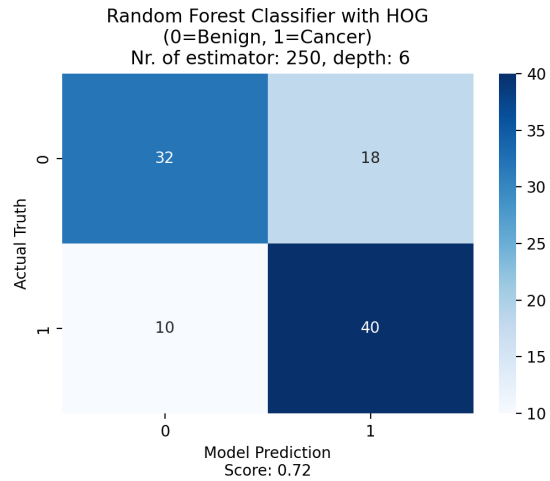


Figure 21: Confusion Matrix for Random Forest Classifier with HOG (Nr. of estimator: 250, depth: 6)

Even though the difference is minor, only 2 patients, the model does not use significantly more resources and power to train the models. Also it is worth mentioning that this 2% might look minor, however if we scale it up for thousands of pictures, essentially thousands of more patients, the 2% reduction does matter! Therefore we conclude that our parameters for the model is 250 estimators and max depth 6. This model correctly predicts 72% of times whether a picture is cancerous or not, 18% of the time falsely predicts a picture being cancerous when it is not and crucially, only 10% of the times predicts a picture benign when it is actually cancerous! We can

conclude that HOG does improve our Random Forest Classifier.

6.2 Local Binary Pattern

[9] [8] [13] Local Binary Pattern, or shortly LBP, is a texture descriptor used in computer vision and image processing. LBP has a wide range of applications including image classification, image segmentation, and pattern recognition. This is due to the texture of an image which can say a lot about a certain qualities of set image. The computational simplicity of the LBP algorithms has made it a option for analyzing in real time and make it an effect tool analyses in general. LBP is usually done and is most efficient in grayscale, and is what we will be using in this paper.

The simple idea behind LBP is for every pixel in an image, you look at the surrounding pixel and assign them a value between zero and one. The center pixel will then get assigned these values so you get a list of binary numbers, hence the name Local Binary Pattern. Then you multiply these values with power values and sum them together. You then do this for every pixel and put them in a histogram where you can analyze the picture. Now let's look into each idea more in depth.

As stated before, we look at every pixel in the picture, and compare it to the surrounding pixels. Since our image is in grayscale each pixel has a value between 0 and 255. We start by look at our center pixels value compared to the pixel in the top left corner of the 3x3 grid, and see if it has a value greater of less then our center pixel's. If it has a greater value than the center pixel's it gets the value 1, and if it is less than the center pixel's it will get assigned a value of 0. Then you do the same for the pixel above the center pixel until you have done it for all surrounding pixel where you have gone clockwise around the center pixel. You are left with an 8 digit binary number. There are $2^8 = 256$ different combinations we can achieve. For example you could end up with a number like "10011010". To get a value you can put in a histogram, we will multiply each of the digits respectively with 2^n where n goes from $n = 0, 1, 2, 3, 4, 5, 6, 7$ and n_1 is for the first pixel and n_8 is for the last. So the first digit of our binary number from before would be $1 \cdot 2^0 = 1$, and the second $0 \cdot 2^1 = 0$, all the way up to the last $0 \cdot 2^7 = 0$. All the numbers are summed together, and for the specific binary number, we get 89. Then the value you get is saved, and you move onto the next pixel until you have done it for all pixel in your picture.

We are now left with a picture where we can see the texture of the image and a histogram. We normalize every values, so the size of the picture does not matter. We are left with a feature-vectors representing texture in our image and we can use it for machine learning tasks [6].

How this might help our project? We believe extracting this feature could improve our Random Forest Classifier. If we look at 2 different pictures, where one is benign and the other is cancerous, we can see how they differ visually.

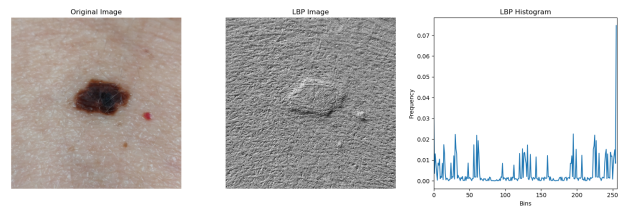


Figure 22: LBP Benign Lesion

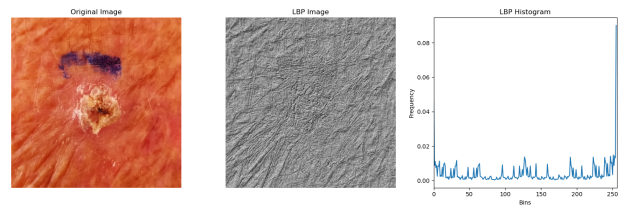


Figure 23: LBP Cancerous Lesion

When we look at the benign lesion in the first figure above, the benign mole is relatively compact, and the surrounding skin areas have a similar texture. The lesion itself is mostly concentrated in one region. As a result, the LBP image shows a more localized abnormal texture, and the histogram contains stronger repeated patterns.

In the case of the cancerous lesion in the second figure above, the LBP does not see one clean contained region. Instead, there are a lot of texture pattern shifts where it goes from smooth skin to rougher textures, and you cannot see clearly where the mole is supposed to be contained. Because LBP looks at the texture changes, it produces a more scattered histogram. This means the histogram is spread across many different bins, showing that there are many different texture patterns rather than one consistent structure.

This is why LBP may be able to help us detect cancer in lesions. It might have a few pitfalls. LBP has a hard time with hair, and pen stripes. It also has

a hard time with texture, if the lesion is on a weird part of the body.

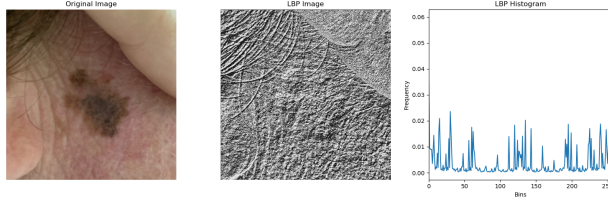


Figure 24: LBP Cancerous Lesion, with Disruption

Looking at the figure above, we can see how the hair, and placement of the lesion can affect the outcome of the LBP data/histogram.

So how does all this perform in our project? It works fine, but not as well as we had hoped. First of all, we chose the Random Forest Classifier as our model of choice. This was due to the amount of data points generated by LBP. There are going to be 256 new features in our dataset. This is discussed in the HOG section, and previously in the Random Forest section.

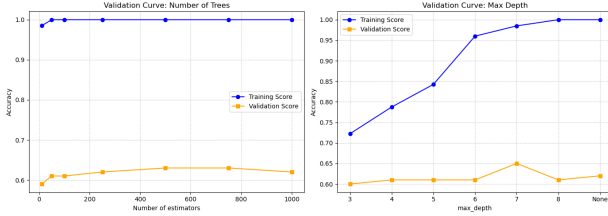


Figure 25: Analysis on number of estimators and max depth for LBP

Looking at the plots, we can clearly see that the highest score for max depth is 7. For the estimator, there are hardly any performance differences, when it is above 50 estimators. We tried a few different hyperparameters to see what performed best. The best results were with either 100 or 500 estimators, both with depth 7.

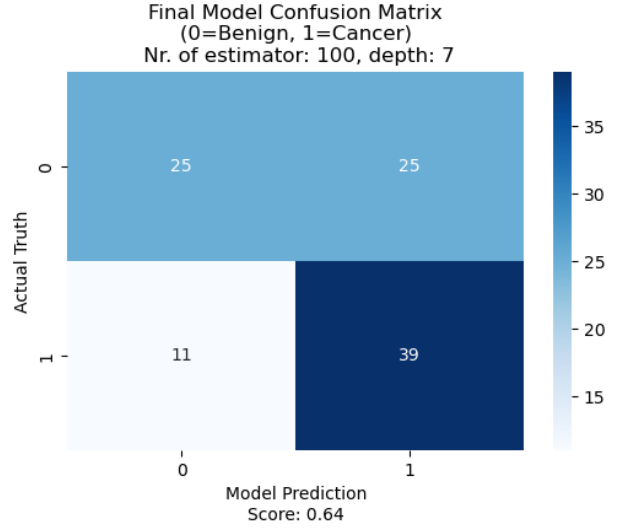


Figure 26: Confusion Matrix for Random Forest Classifier with LBP (Nr. of estimator: 100, depth: 7)

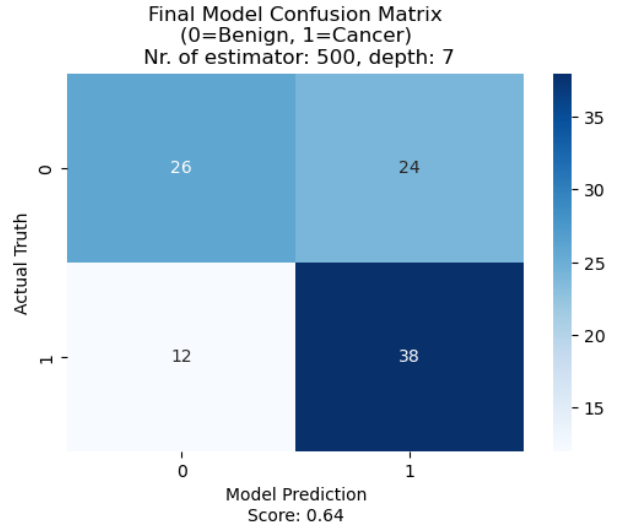


Figure 27: Confusion Matrix for Random Forest Classifier with LBP (Nr. of estimator: 500, depth: 7)

Both models have 0.64 accuracy score, and they have a very similar distributions of false negatives and positives. Still there was not a notable improvement when using LBP and there are too many false positives compared to HOG where it is worth it to use.

What if we combine the two? We combined the two features, HOG and LBP. Now we have a dataset that has around 2000 features. Following the same steps to test the Random Forest's accuracy, we get the following figures.

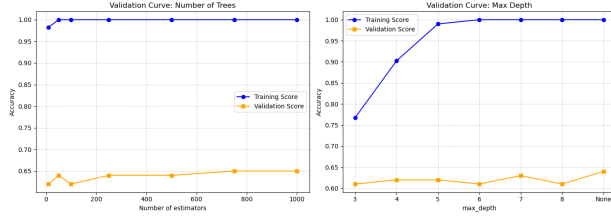


Figure 28: Analysis on number of estimators and max depth for HOG and LBP

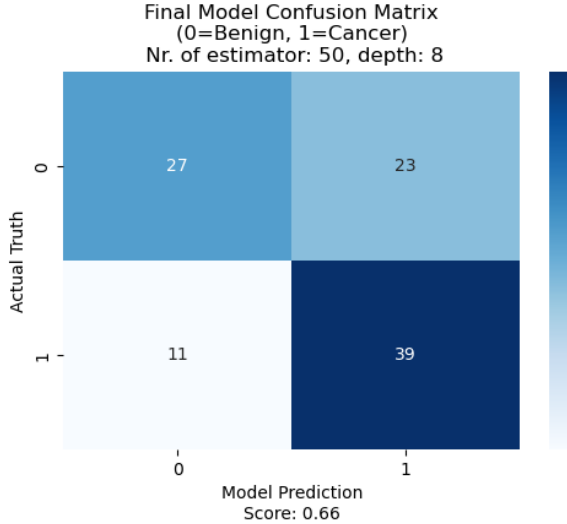


Figure 29: Confusion Matrix for Random Forest Classifier with HOG and LBP (Nr. of estimator: 50, depth: 8)

We tried many different amount of estimators and depths. The best one had an accuracy score of 0.66, with a decent number of true negatives and positives. There are still a lot of false positives, which we do not want. We suspect that LBP has a hard time with the pictures because of the variety of the pictures. If the quality of the picture is not as good, LBP loses accuracy because fewer pixels mean less data, and when making the binary code, it might result in a rougher average. As mentioned earlier, the placement of the lesion could have an impact on the data extracted by LBP.

Some of these problems may have been avoided by looking only at the lesion, and maybe a little more of the surrounding skin. For the quality of the image, perhaps we could have upsampled it. The problem with upscaling, could be that patterns could show up that would not have been there. The running time of LBP would also take longer since there would be more pixels. We chose to focus only on HOG since it performs better, than both of them together.

7 Conclusion

In this project, we investigated whether handcrafted image features could be used to classify skin lesions as cancerous or benign. We extracted features based on asymmetry, border irregularity and colour variation, inspired by the ABC method used by dermatologists.

We tested different machine learning models, mainly KNN and Random Forest Classifier to see how well the extracted features performed. We found that Random Forest generally performed better, especially because it handled nonlinear relationships and high-dimensional data better than KNN.

We also tested SMOTE and saw that generating synthetic cancerous samples improved the models performance and reduced the number of false negatives. Since false negatives are especially important in medical imaging, this was one of the main things we focused on throughout the project. Finally we investigated LBP and HOG feature extraction methods. While LBP showed some promising result, HOG performed the best overall. By combining HOG with random Forest Classifier, we achieved our best model with an accuracy score around 72% while also reducing the amount of false negatives.

7.1 What could we improve?

Although we achieved some promising results throughout the project, there are still several things that could be improved.

After extracting the features and evaluating the data, we realised we did not removed the images with missing masks which could have resulted in a smaller dataset. Also the images with unfitted masks could have changed the overall process of feature extraction.

One of the biggest limitations was the size of the dataset. Even though we did not follow our initial plan to remove all images with missing or unfitted masks, we have ran into computational limitations and that is why we only used about 500 images on many of our experiments. A larger dataset would likely improve the models ability to generalise and produce more reliable results.

Another challenge was the quality of the images. Since the pictures were taken using smartphones, there are many differences in lightning, image quality, angle and distance from the lesion which sometimes affected our feature extractions methods (especially LBP). More controlled camera conditions and lightning could help produce more consistent images and therefore improve the overall models performance.

As mention before, we mainly focused on accuracy, recall and false negatives when evaluating the models. In the future, we could include more more

performance metrics such as F1-score, which balances precision and recall, making it useful when both false negatives and false positives are important.

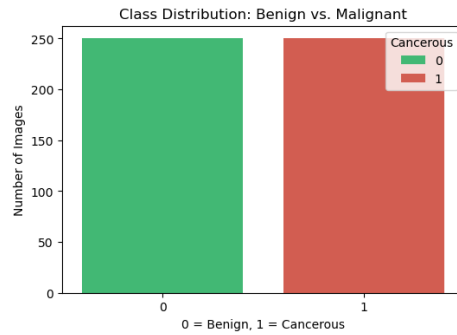
In addition, we could also take a look at more advanced models such as CNNs (Convolutional Neural Networks). Compared to the models we have used, CNNs are able to automatically learn important visual patterns directly from the images, instead of relying on handcrafted feature extraction methods such as asymmetry, border compactness, HOG and LBP.

References

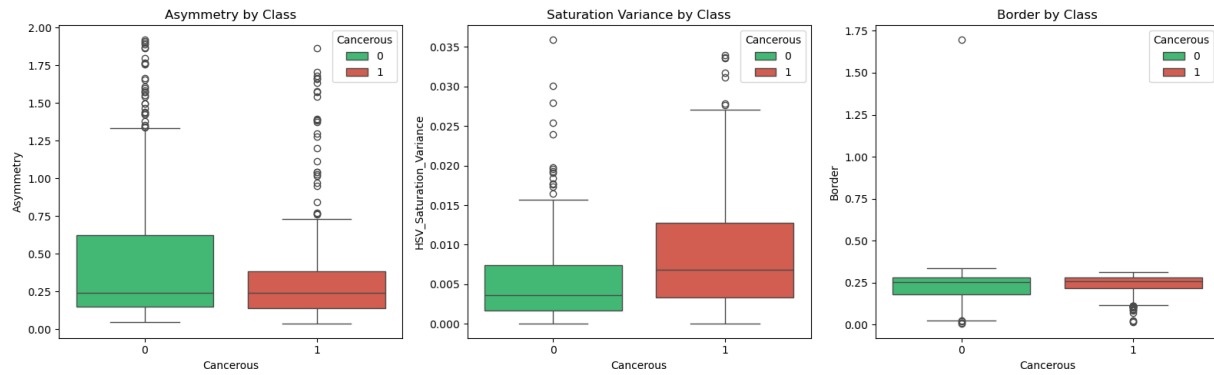
- [1] Geeks for geeks. “Histogram of Oriented Gradients”. In: *Geeks for geeks* (2025). URL: <https://www.geeksforgeeks.org/computer-vision/histogram-of-oriented-gradients/>.
- [2] geeksforgeeks. “SMOTE for Imbalanced Classification with Python”. In: *geeksforgeeks* (10 Feb, 2026). URL: <https://www.geeksforgeeks.org/machine-learning/smote-for-imbalanced-classification-with-python/>.
- [3] Sydney Ghazarian. “Melanoma skin cancer cases hit a record high in the UK”. In: *Cancer Research UK* (2026). URL: <https://news.cancerresearchuk.org/2026/05/22/skin-cancer-cases-hit-record-high-in-the-uk/>.
- [4] M.D. Mika Tabata. “How can you tell if a mole is cancerous?” In: *The University of Texas MD Anderson Cancer Center* (2025). URL: <https://www.mdanderson.org/cancerwise/how-can-you-tell-if-a-mole-is-cancerous.h00-159778023.html#>.
- [5] World Health Organization. “Skin Cancer”. In: *World Health Organization* (). URL: <https://www.iarc.who.int/cancer-type/skin-cancer/>.
- [6] P.Umamaheswari et al. “SKIN CANCER DETECTION USING LOCAL BINARY PATTERN AND SVM CLASSIFIER”. In: *International Journal of Computer Research and Technology (IJCRT)* (2024). URL: <https://ijcrt.org/papers/IJCRT24A3130.pdf>.
- [7] Andre G.C. Pacheco et al. “PAD-UFES-20: A skin lesion dataset composed of patient data and clinical images collected from smartphones”. In: *Data in Brief* (2020). DOI: <https://doi.org/10.1016/j.dib.2020.106221>. URL: <https://www.sciencedirect.com/science/article/pii/S235234092031115X>.
- [8] M. Pietikäinen. “Local Binary Patterns”. In: *Scholarpedia* 5.3 (2010). revision #203567, p. 9775. DOI: [10.4249/scholarpedia.9775](https://doi.org/10.4249/scholarpedia.9775). URL: www.scholarpedia.org/article/Local_Binary_Patterns.
- [9] Ahmedur Rahman Shovon. “Local Binary Patterns implementation using Python 3”. In: *GitHub* (Mar 3, 2017). URL: https://github.com/arsho/local_binary_patterns.
- [10] Jared Wilber Brent Werness. “The KNN Classifier”. In: *MLU-explain* (). URL: <https://mlu-explain.github.io/bias-variance/>.
- [11] Wikipedia. “Histogram of oriented gradients”. In: *Histogram of oriented gradients* (2025). URL: https://en.wikipedia.org/wiki/Histogram_of_oriented_gradients.
- [12] Jenny Yeon Jared Wilber. “The Random Forest Algorithm”. In: *MLU-explain* (). URL: <https://mlu-explain.github.io/random-forest/>.
- [13] B. S. Bigham Z. Sedaghatjoo H. Hosseinzadeh. “Local Binary Pattern (LBP) Optimization for Feature Extraction”. In: *arXiv* (July 29, 2024). URL: <https://arxiv.org/pdf/2407.18665v1>.

8 Appendix

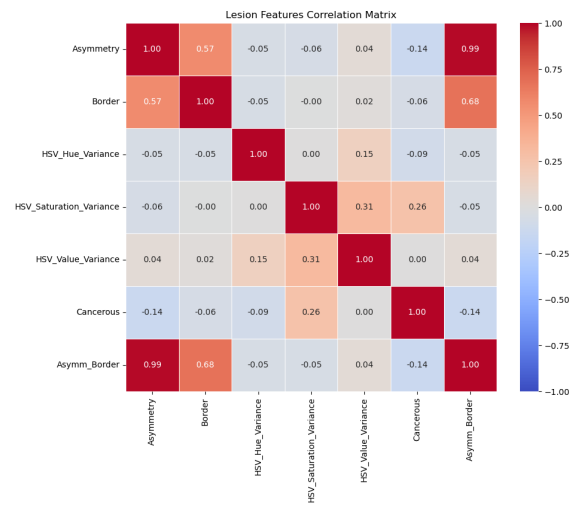
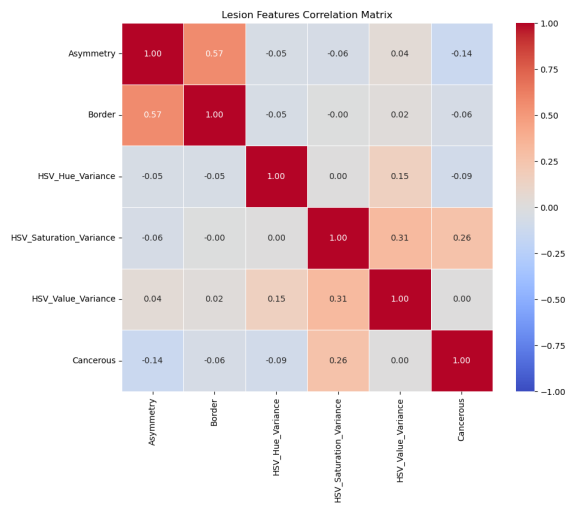
8.0.1 Bar plot



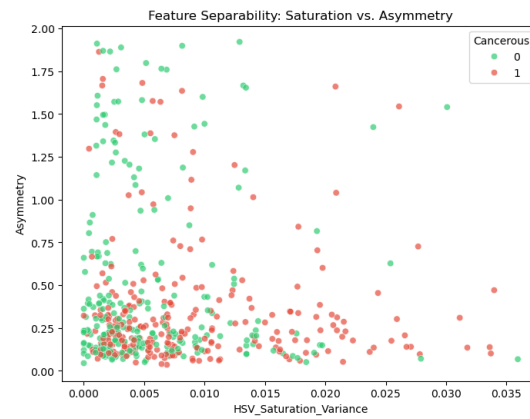
8.0.2 Box-plot



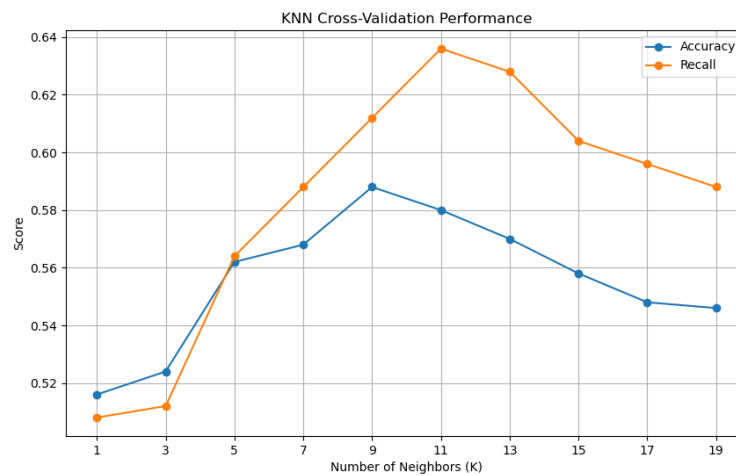
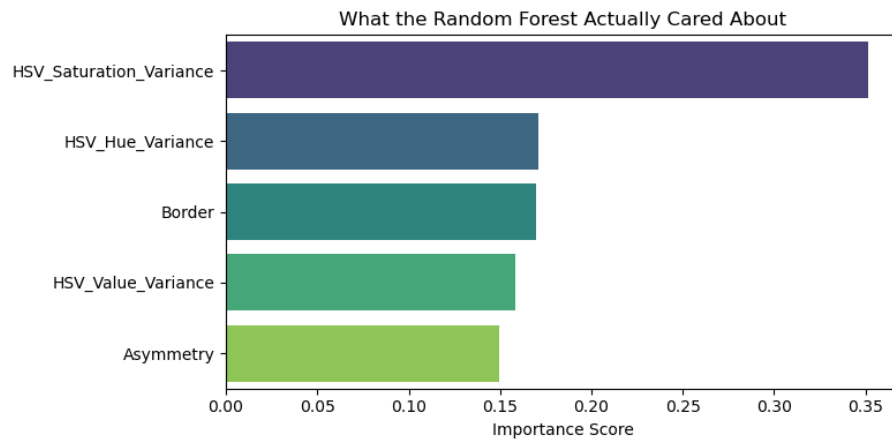
8.0.3 Correlation Matrix

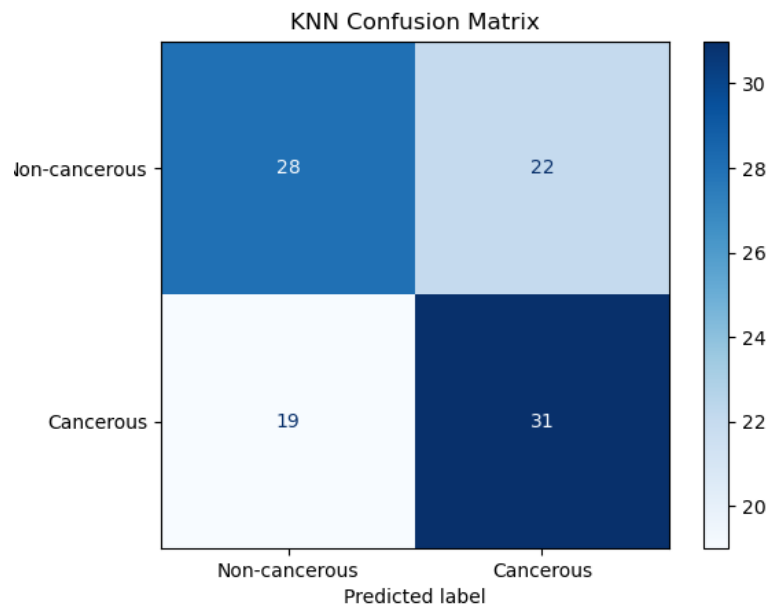
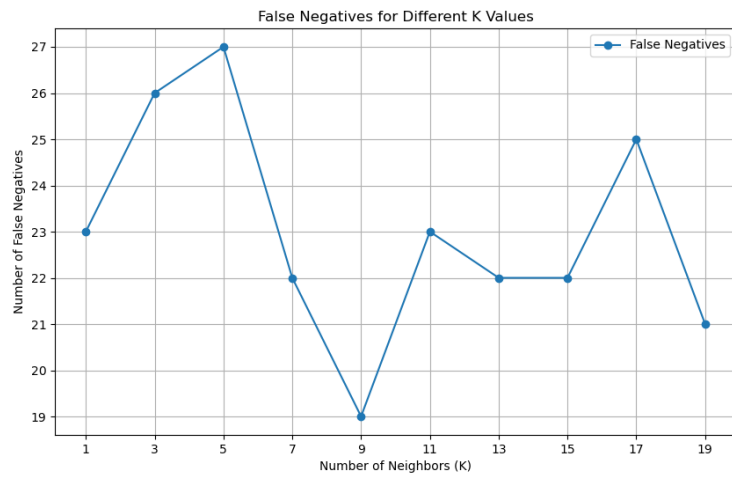


8.0.4 Scatter-plot

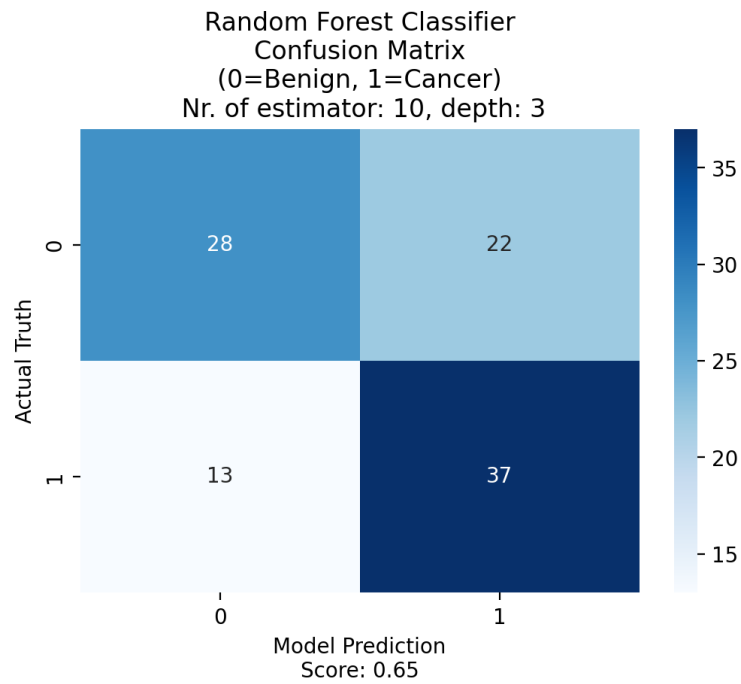


8.0.5 Random Forest





8.1 Estimator and max depth on Random Forest Classifier



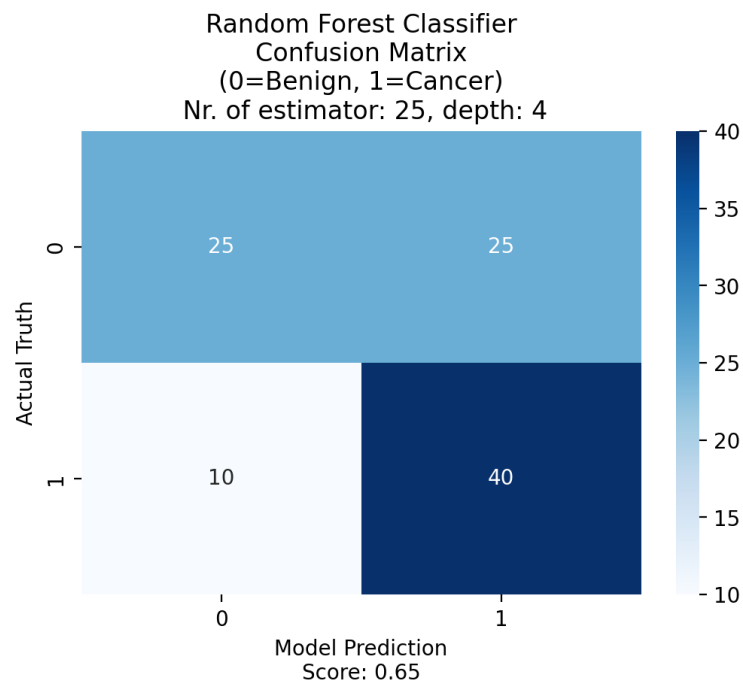
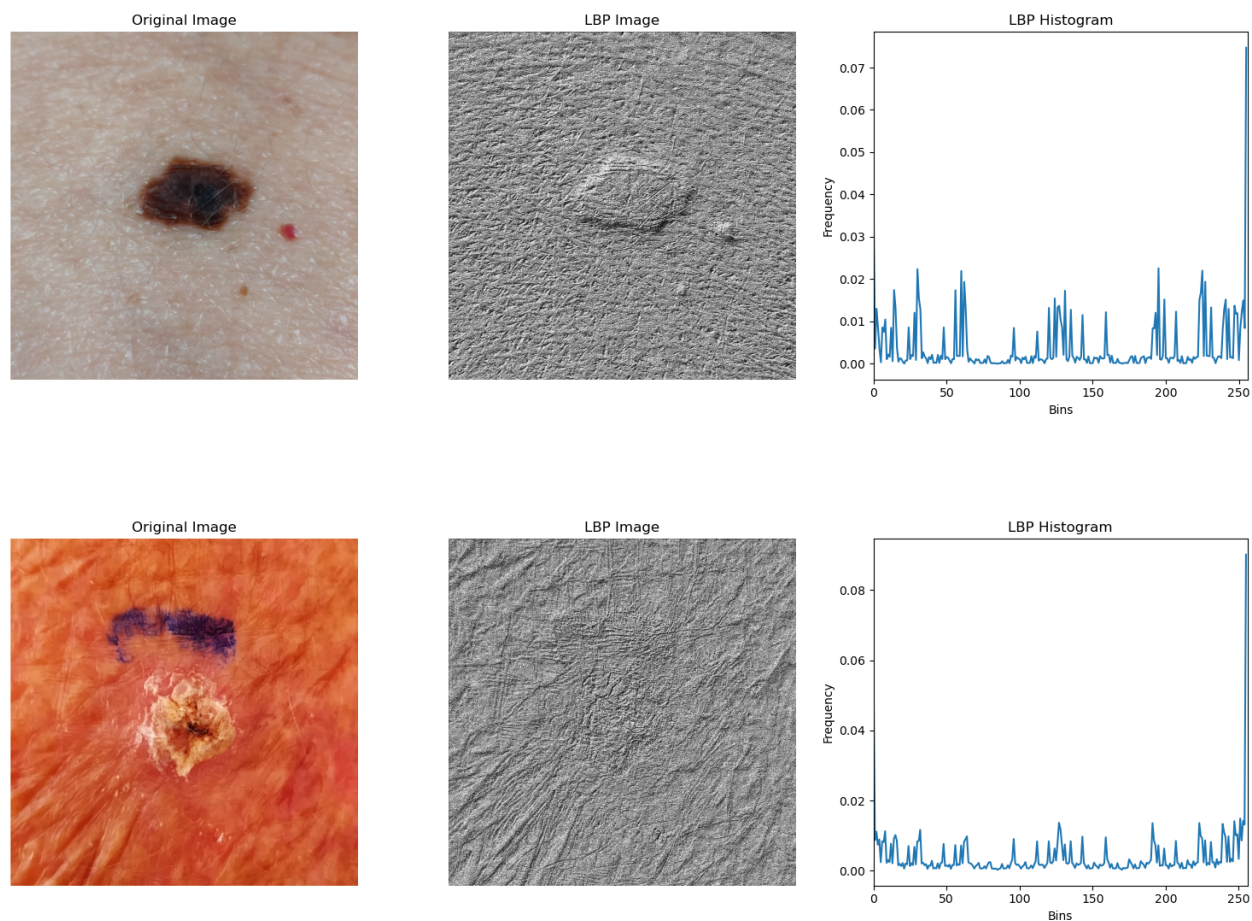
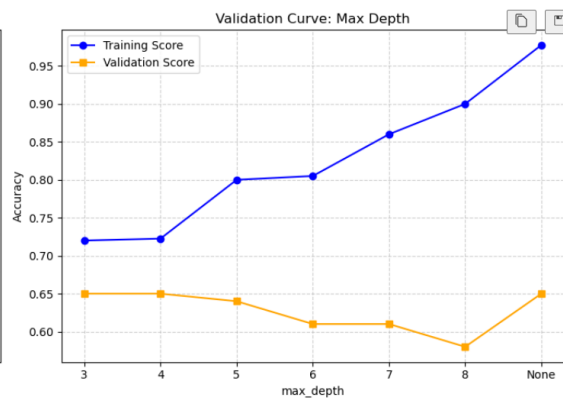
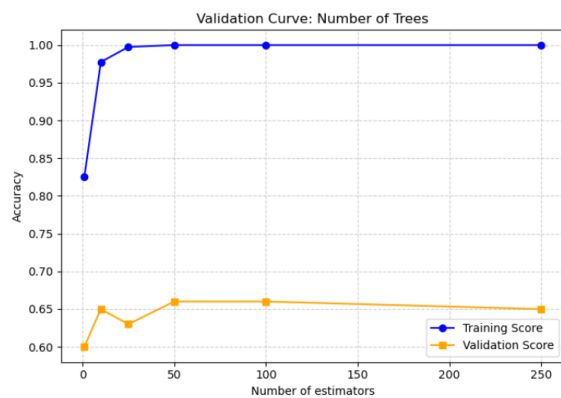
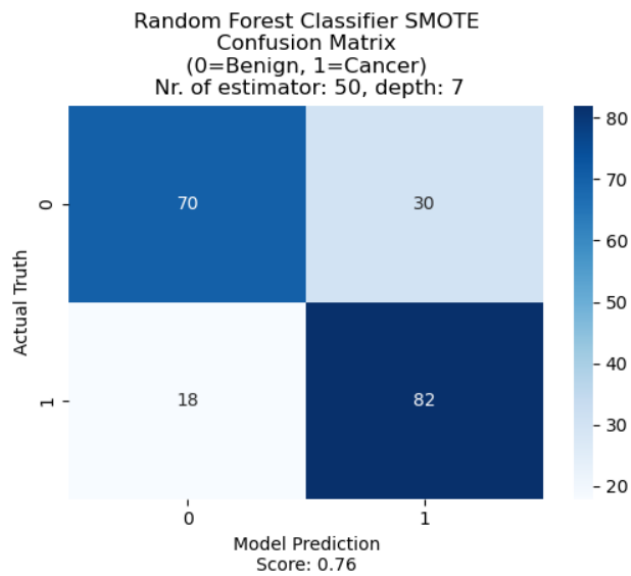
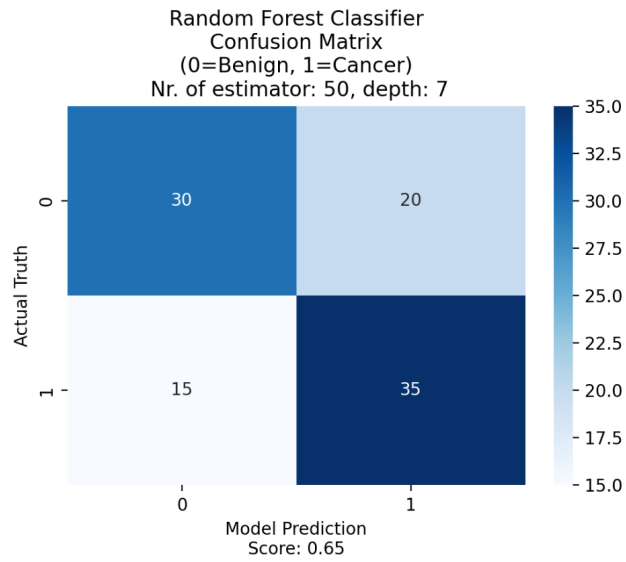
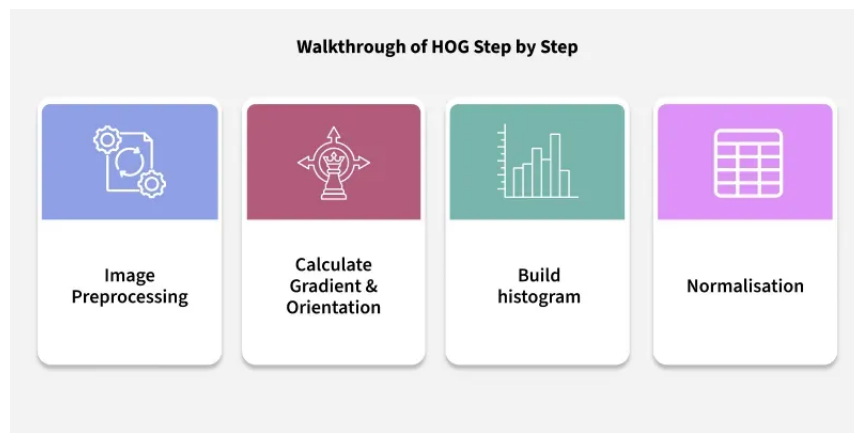
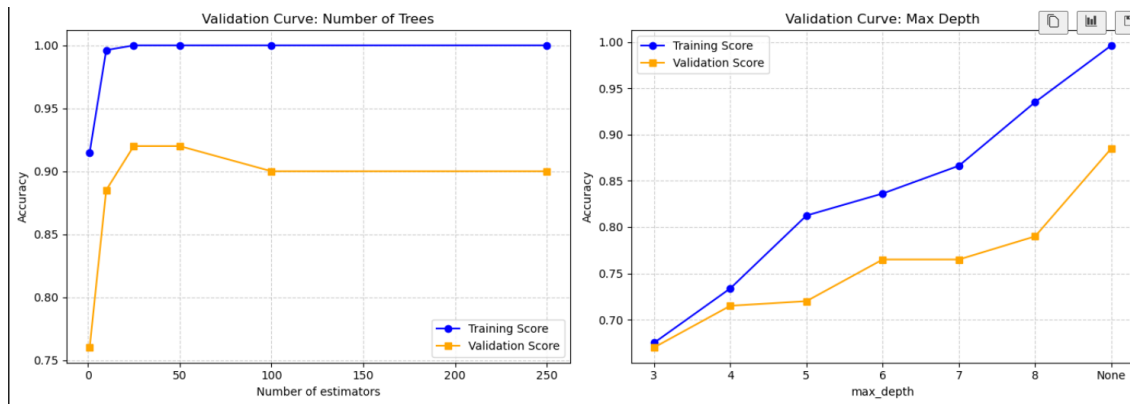


Figure 30: Confusion Matrix for Random Forest Classifier (Nr. of estimator: 25, depth: 4)



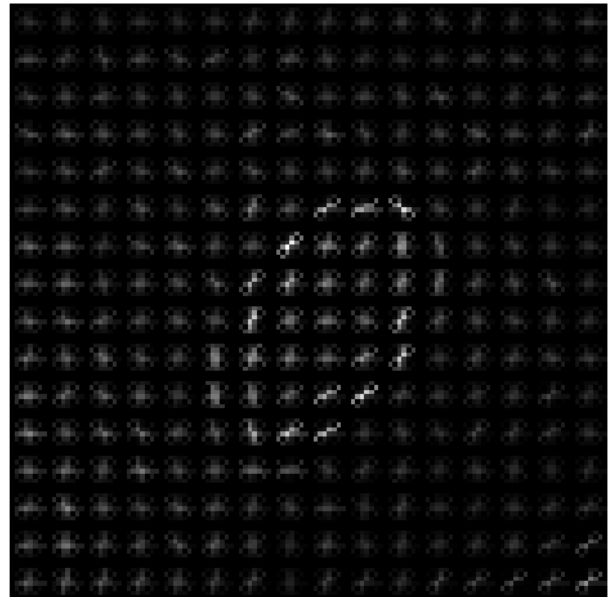




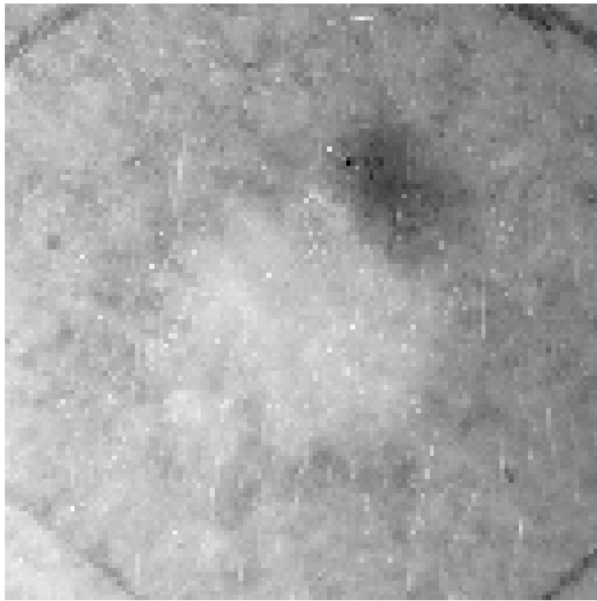
Original Grayscale Lesion



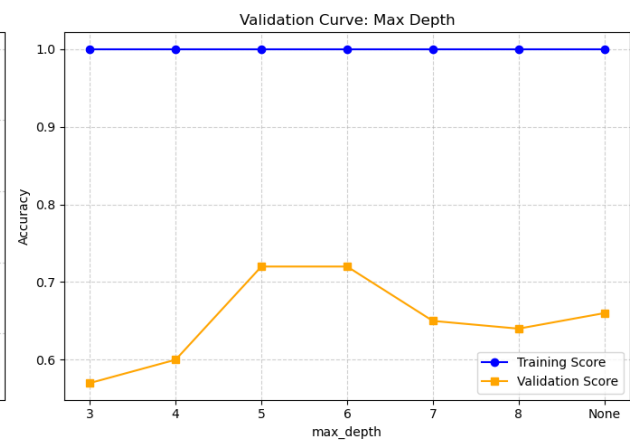
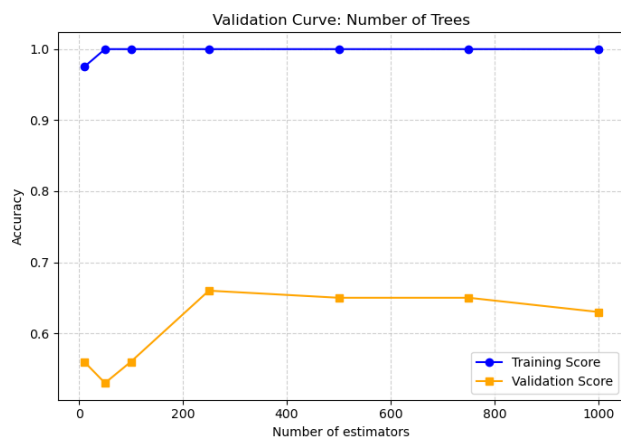
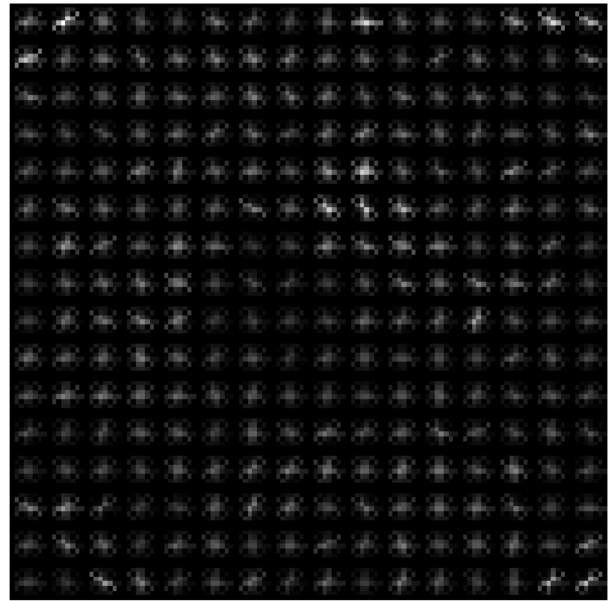
HOG Texture Map

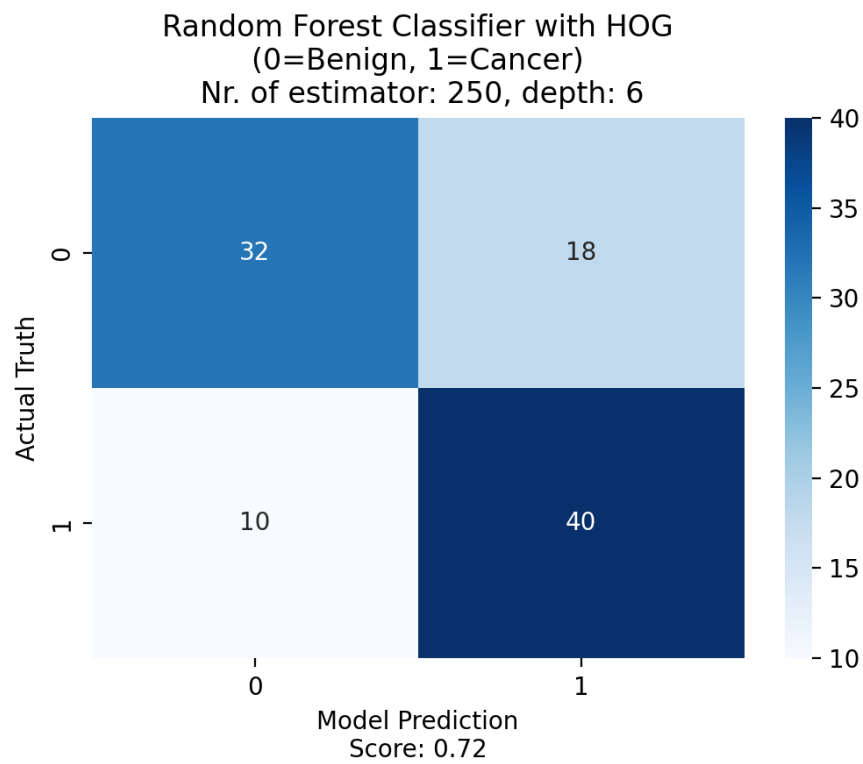
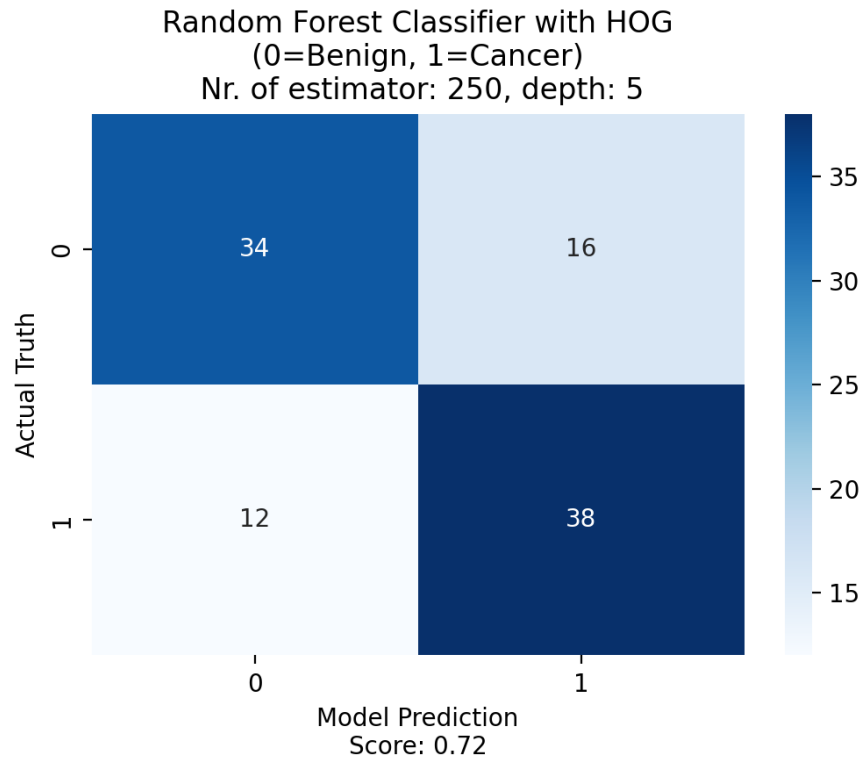


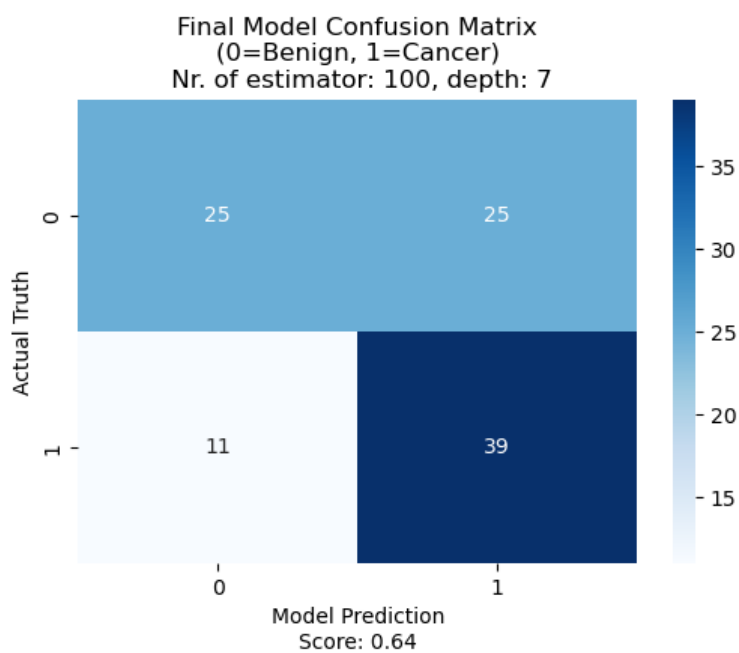
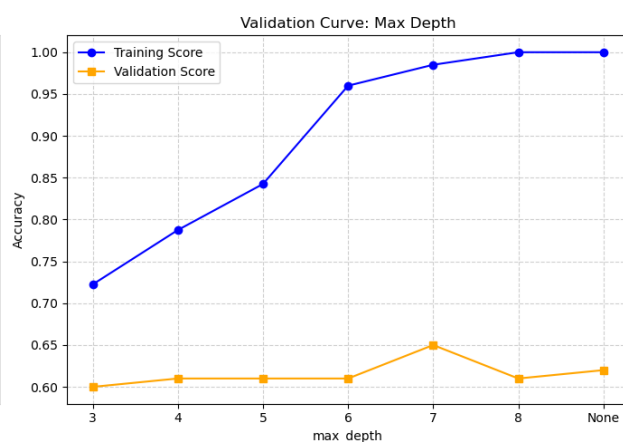
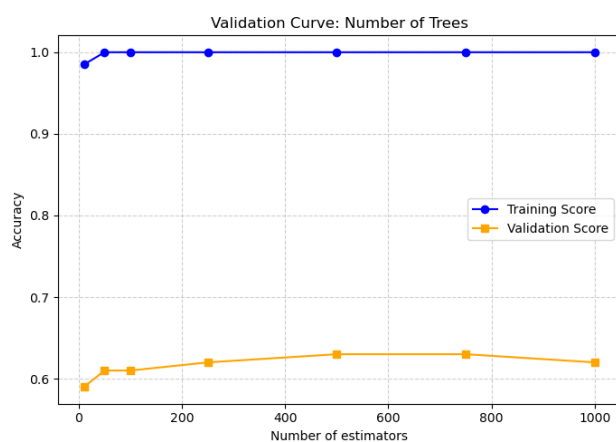
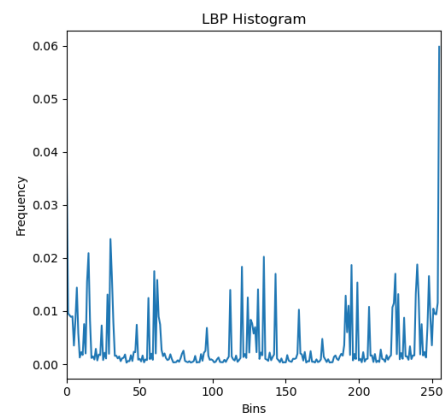
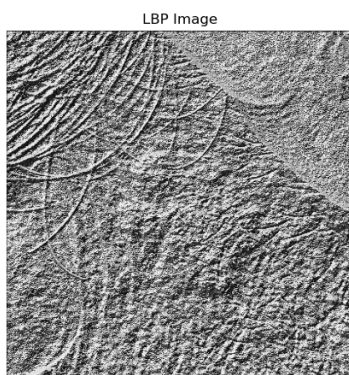
Original Grayscale Lesion



HOG Texture Map







Final Model Confusion Matrix
(0=Benign, 1=Cancer)
Nr. of estimator: 500, depth: 7

